

BÀI TẬP 01: TỔNG QUAN VỀ MÔN HỌC

Yêu cầu: Các anh (chị) hãy trình bày báo cáo trả lời các câu hỏi bên dưới. Báo cáo có trang bìa trình bày ghi rõ họ tên, mã sinh viên.

TỔNG QUAN VỀ MÔN HỌC

1. Tổng quan về kiến trúc phần mềm

Kiến trúc hệ thống đóng vai trò quan trọng như nền móng của một tòa nhà trong lĩnh vực phát triển phần mềm. Kiến trúc không chỉ quyết định cách các thành phần phần mềm tương tác mà còn ảnh hưởng trực tiếp đến khả năng mở rộng, bảo trì và hiệu suất của hệ thống.

Chúng ta sẽ khám phá ba loại kiến trúc phổ biến: Monolithic, Modular Monolithic, và Microservices. Mỗi loại kiến trúc mang lại những lợi ích và thách thức riêng, phù hợp với những bài toán và quy mô khác nhau. Ngoài ra, các nguyên tắc thiết kế như KISS, YAGNI, và Separation of Concerns sẽ được trình bày để giúp bạn xây dựng một hệ thống không chỉ hiệu quả mà còn dễ dàng thích nghi với sự thay đổi.

- **Các kiểu kiến trúc chính:**

- **Monolithic Architecture:** Tất cả các chức năng của ứng dụng nằm trong một khối duy nhất, dễ triển khai nhưng khó mở rộng.
- **Modular Monolithic Architecture:** Phân tách thành các mô-đun độc lập bên trong một khối ứng dụng, giúp tăng tính tổ chức và bảo trì.
- **Microservices Architecture:** Chia ứng dụng thành các dịch vụ nhỏ, độc lập, dễ mở rộng và duy trì nhưng phức tạp hơn khi triển khai.

- **Nguyên tắc thiết kế:**

- **KISS (Keep It Simple, Stupid):** Thiết kế càng đơn giản càng tốt.
- **YAGNI (You Aren't Gonna Need It):** Không thêm các tính năng không cần thiết.
- **Separation of Concerns (SoC):** Phân tách trách nhiệm giữa các thành phần.
- **SOLID Principles:** Giúp thiết kế có tính mở rộng, dễ bảo trì.

2. Microservices Architecture

Nhu cầu xây dựng các hệ thống linh hoạt, dễ mở rộng và duy trì đã đưa Microservices Architecture trở thành một trong những xu hướng thiết kế phổ biến nhất. Không giống như kiến trúc monolithic truyền thống, microservices phân chia ứng dụng thành các dịch vụ nhỏ, độc lập, giúp tối ưu hóa hiệu năng, giảm độ phức tạp trong phát triển và triển khai.

Phần này sẽ đi sâu vào các đặc điểm nổi bật của microservices, từ khả năng mở rộng độc lập đến cách tiếp cận triển khai linh hoạt. Chúng ta cũng sẽ tìm hiểu các mô hình giao tiếp giữa các dịch vụ như RESTful API, gRPC, và WebSocket, đồng thời khám phá những pattern phổ biến trong giao tiếp và triển khai microservices. Ngoài ra, những lợi ích cũng như thách thức khi áp dụng microservices sẽ được phân tích chi tiết.

- **Đặc điểm chính:**

- Dịch vụ độc lập, triển khai linh hoạt, hỗ trợ công nghệ đa dạng.
- Khả năng mở rộng theo chiều ngang, phù hợp với khối lượng lớn yêu cầu người dùng.
- Cách ly lỗi, dễ phục hồi trong các tình huống thất bại.

- **Giao tiếp giữa các dịch vụ:**

- **HTTP-based RESTful API:** Phổ biến, dễ triển khai.
- **gRPC:** Giao tiếp hiệu suất cao, dùng mã hóa nhị phân.

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI ©DNTAI

- **WebSocket:** Hỗ trợ giao tiếp thời gian thực, hai chiều.
- **Các pattern giao tiếp:**
 - **Gateway Routing Pattern:** Định tuyến các yêu cầu qua API Gateway.
 - **Gateway Aggregation Pattern:** Tổng hợp dữ liệu từ nhiều dịch vụ.
 - **Backends for Frontends (BFF):** Tối ưu hóa giao diện với API dành riêng.

3. Quản lý dữ liệu trong microservices

Dữ liệu là trái tim của mọi hệ thống phần mềm, và trong kiến trúc microservices, việc quản lý dữ liệu trở nên phức tạp hơn bao giờ hết. Thay vì sử dụng một cơ sở dữ liệu chung, mỗi microservice thường sở hữu cơ sở dữ liệu riêng, điều này mang lại sự độc lập nhưng cũng đi kèm với những thách thức về đồng bộ và tính nhất quán.

Trong phần này, chúng ta sẽ khám phá các kỹ thuật và pattern quan trọng để quản lý dữ liệu trong microservices. Từ Database-per-Service Pattern đến các phương pháp Data Partitioning như phân mảnh ngang, phân mảnh dọc, và phân mảnh theo chức năng, những cách tiếp cận này sẽ giúp tối ưu hóa hiệu suất và khả năng mở rộng.

Bên cạnh đó, các khái niệm như CQRS (Command Query Responsibility Segregation), Event Sourcing Pattern, và quản lý giao dịch phân tán với Saga Pattern sẽ được trình bày để giải quyết các bài toán phức tạp liên quan đến truy vấn, ghi dữ liệu, và tính nhất quán trong môi trường phân tán.

- **Database-per-Service Pattern:** Mỗi dịch vụ sở hữu một cơ sở dữ liệu riêng, đảm bảo tính độc lập và cách ly dữ liệu.
- **Data Partitioning:**
 - **Horizontal Partitioning:** Phân chia theo hàng.
 - **Vertical Partitioning:** Phân chia theo cột.
 - **Functional Partitioning:** Phân chia theo chức năng.
- **CQRS (Command Query Responsibility Segregation):** Phân tách giữa xử lý truy vấn (query) và xử lý lệnh (command), tăng hiệu suất.
- **Event Sourcing Pattern:**
 - Lưu trữ toàn bộ các sự kiện thay đổi trạng thái, không lưu trạng thái cuối.
 - Hỗ trợ phục hồi và tái hiện trạng thái hệ thống.
- **Distributed Transactions:**
 - **Saga Pattern:** Chia giao dịch lớn thành các bước nhỏ, dễ quản lý và phục hồi khi thất bại.
 - **Transactional Outbox Pattern:** Đảm bảo tính nhất quán khi ghi dữ liệu ra cơ sở dữ liệu.
 - **Change Data Capture (CDC):** Ghi nhận các thay đổi dữ liệu để đồng bộ hóa.

4. Resilience và triển khai hệ thống

Trong môi trường hệ thống phân tán như microservices, các lỗi không chỉ khó tránh khỏi mà còn có thể gây ra hiệu ứng domino làm gián đoạn toàn bộ dịch vụ. Vì vậy, khả năng chịu lỗi (resilience) và phương pháp triển khai hệ thống hiệu quả là hai yếu tố cốt lõi để đảm bảo tính ổn định và liên tục của hệ thống.

Phần này tập trung vào các resilience patterns như Retry, Circuit Breaker, Bulkhead, và Timeout, giúp giảm thiểu tác động của lỗi và đảm bảo dịch vụ hoạt động trơn tru ngay cả khi gặp sự cố. Ngoài ra, việc giám sát và quan sát hệ thống thông qua các công cụ như Prometheus, Grafana, và Elastic Stack cũng sẽ được giới thiệu, mang lại khả năng theo dõi và phân tích toàn diện.

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI ©DNTAI

Bên cạnh đó, chúng ta sẽ tìm hiểu các chiến lược triển khai như Blue-Green Deployment, Rolling Deployment, và Canary Deployment – những cách tiếp cận hiện đại để cập nhật hệ thống mà không gây gián đoạn dịch vụ. Cuối cùng, vai trò của Containers và Orchestrators như Docker và Kubernetes trong việc tự động hóa và quản lý quy trình triển khai sẽ được làm rõ.

- **Resilience Patterns:**
 - **Retry:** Thử lại khi gặp lỗi tạm thời.
 - **Circuit Breaker:** Ngắt kết nối nếu phát hiện lỗi thường xuyên.
 - **Bulkhead:** Phân vùng tài nguyên để giảm thiểu ảnh hưởng của lỗi.
 - **Timeout:** Giới hạn thời gian chờ phản hồi.
- **Giám sát và quan sát hệ thống:**
 - **Prometheus:** Thu thập và giám sát chỉ số hiệu năng.
 - **Grafana:** Trực quan hóa dữ liệu giám sát.
 - **Elastic Stack:** Phân tích nhật ký hệ thống.
- **Chiến lược triển khai:**
 - **Blue-Green Deployment:** Giảm thời gian chết bằng cách triển khai vào môi trường mới.
 - **Rolling Deployment:** Cập nhật lần lượt từng phần của hệ thống.
 - **Canary Deployment:** Triển khai thử nghiệm trên một phần nhỏ trước khi áp dụng toàn hệ thống.

5. Các bài toán thực tế và giải pháp

Khi xây dựng và vận hành các hệ thống microservices, những bài toán thực tế thường xuyên xuất hiện, từ quản lý giao tiếp giữa các dịch vụ, tối ưu hóa hiệu suất, đến xử lý dữ liệu lớn và đảm bảo tính nhất quán trong giao dịch phân tán. Để giải quyết những thách thức này, các kiến trúc sư phần mềm cần áp dụng các giải pháp thực tiễn, linh hoạt và hiệu quả.

Phần này sẽ đi sâu vào các bài toán phổ biến trong microservices:

- **Giao tiếp giữa các dịch vụ:** Các vấn đề về độ trễ, giao tiếp đồng bộ và số lượng API calls sẽ được giải quyết thông qua các pattern như Asynchronous Messaging và Service Aggregator.
- **Hiệu suất hệ thống:** Sử dụng caching phân tán, sharding cơ sở dữ liệu, và tối ưu hóa truy vấn để đảm bảo hệ thống hoạt động ổn định dưới tải cao.
- **Dữ liệu lớn và event-driven architecture:** Quản lý hàng triệu sự kiện mỗi giây bằng cách áp dụng kiến trúc event-driven và stream processing.
- **Giao dịch phân tán:** Đảm bảo tính nhất quán và phục hồi khi thất bại với các giải pháp như Saga Pattern, CQRS, và Event Sourcing.

Ngoài ra, phần này cũng nhấn mạnh tầm quan trọng của việc lựa chọn giải pháp phù hợp với nhu cầu kinh doanh và quy mô hệ thống, thay vì cố gắng áp dụng công nghệ mới một cách không cần thiết.

- **Giao tiếp giữa các dịch vụ:**
 - **Vấn đề:** Giao tiếp đồng bộ giữa các dịch vụ gây ra độ trễ cao.
 - **Giải pháp:** Sử dụng giao tiếp bất đồng bộ với Event-driven Architecture.
- **Hiệu suất hệ thống:**
 - **Vấn đề:** Nhiều truy vấn tới cơ sở dữ liệu làm giảm hiệu suất.
 - **Giải pháp:** Dùng caching phân tán, sharding cơ sở dữ liệu.
- **Quản lý dữ liệu lớn:**
 - **Vấn đề:** Xử lý hàng triệu sự kiện mỗi giây.
 - **Giải pháp:** Sử dụng kiến trúc Event-driven với stream processing.
- **Quản lý giao dịch phân tán:**

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI ©DNTAI

- Vấn đề: Đảm bảo tính nhất quán trong giao dịch trên nhiều dịch vụ.
- Giải pháp: Áp dụng Saga Pattern và CQRS để quản lý giao dịch.

6. Tư duy thiết kế

Trong phát triển phần mềm, công nghệ và công cụ luôn thay đổi, nhưng một tư duy thiết kế đúng đắn sẽ giúp bạn xây dựng những hệ thống không chỉ đáp ứng yêu cầu hiện tại mà còn có khả năng mở rộng, thích nghi với tương lai. Tư duy thiết kế không đơn giản là lựa chọn công nghệ hay kiến trúc, mà là cách bạn phân tích, đánh giá và đưa ra các quyết định dựa trên yêu cầu thực tiễn và mục tiêu kinh doanh.

Phần này sẽ nhấn mạnh vai trò của các nguyên tắc thiết kế tập trung vào yêu cầu kinh doanh:

- Tránh over-engineering: Không nên sử dụng các công nghệ phức tạp hoặc các kiến trúc không cần thiết, đặc biệt khi chúng không mang lại giá trị thực tế.
- Dựa trên yêu cầu chức năng và phi chức năng: Các quyết định thiết kế cần dựa trên số lượng người dùng, thời gian đáp ứng, mức độ chấp nhận rủi ro, và khả năng mở rộng.
- Tập trung vào mục tiêu dài hạn: Mặc dù cần giải quyết các vấn đề trước mắt, thiết kế nên hướng tới khả năng phát triển trong tương lai.

Ngoài ra, phần này sẽ đề cập đến tầm quan trọng của việc cân bằng giữa tính phức tạp của hệ thống và giá trị kinh doanh mà nó mang lại. Một thiết kế tốt không chỉ giải quyết được các vấn đề kỹ thuật mà còn đóng góp vào sự thành công chung của doanh nghiệp.

- **Thiết kế dựa trên yêu cầu kinh doanh:**
 - Tránh thiết kế quá phức tạp (over-engineering).
 - Ưu tiên các tính năng đáp ứng trực tiếp yêu cầu kinh doanh.
- **Định hướng mở rộng:**
 - Thiết kế ban đầu cần linh hoạt, dễ mở rộng.
 - Xây dựng hệ thống đáp ứng các chỉ tiêu như số lượng người dùng đồng thời, thời gian đáp ứng yêu cầu.

TRẮC NGHIỆM KIẾN THỨC

Kiến thức cơ bản về kiến trúc phần mềm

1. Kiến trúc nào dưới đây không thuộc các loại kiến trúc phần mềm phổ biến?
 - A. Monolithic Architecture
 - B. Layered Architecture
 - C. Modular Monolithic Architecture
 - D. Quantum Architecture
2. Nguyên tắc "YAGNI" đề cập đến điều gì?
 - A. Không thêm các tính năng mà bạn sẽ không cần đến
 - B. Thiết kế kiến trúc theo mô-đun nhỏ
 - C. Không phụ thuộc vào công nghệ cụ thể
 - D. Luôn áp dụng mô hình microservices
3. Khi nào nên sử dụng kiến trúc monolithic thay vì microservices?
 - A. Khi hệ thống cần khả năng mở rộng cao
 - B. Khi ứng dụng nhỏ và ít thay đổi
 - C. Khi cần xử lý hàng triệu yêu cầu mỗi giây
 - D. Khi có nhiều đội phát triển độc lập

Thiết kế và triển khai microservices

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI ©DNTAI

4. Trong kiến trúc microservices, mô hình "Database-per-Service Pattern" đề xuất:
 - A. Một cơ sở dữ liệu chung cho tất cả các dịch vụ
 - B. Mỗi dịch vụ có cơ sở dữ liệu riêng của mình
 - C. Chỉ sử dụng cơ sở dữ liệu NoSQL
 - D. Tích hợp dữ liệu bằng cách sử dụng API Gateway
5. Lợi ích chính của sử dụng gRPC trong microservices là gì?
 - A. Giao tiếp không đồng bộ
 - B. Hỗ trợ truyền dữ liệu nhanh với mã hóa nhị phân
 - C. Giao tiếp hai chiều thời gian thực
 - D. Đơn giản hóa thiết kế RESTful
6. Pattern nào dưới đây được sử dụng để xử lý truy vấn dữ liệu và lệnh ghi riêng biệt trong microservices?
 - A. Event Sourcing
 - B. CQRS (Command Query Responsibility Segregation)
 - C. Materialized View Pattern
 - D. Database Sharding Pattern

Xử lý giao tiếp và dữ liệu

7. Pattern nào giúp giảm độ trễ trong giao tiếp giữa các microservices?
 - A. Service Aggregator Pattern
 - B. Publish/Subscribe Messaging
 - C. Backends for Frontends Pattern
 - D. API Gateway Pattern
8. "Event Sourcing Pattern" hoạt động như thế nào?
 - A. Lưu trữ trạng thái hiện tại của hệ thống trong cơ sở dữ liệu
 - B. Lưu trữ tất cả các sự kiện thay đổi trạng thái hệ thống
 - C. Tạo các bản sao dữ liệu để truy vấn nhanh hơn
 - D. Kết hợp nhiều dịch vụ thành một giao tiếp duy nhất
9. Trong mô hình giao dịch phân tán, SAGA Pattern sử dụng:
 - A. Giao dịch nguyên tử trên tất cả các dịch vụ
 - B. Các bước nhỏ độc lập với khả năng khôi phục khi thất bại
 - C. Một cơ sở dữ liệu duy nhất để đảm bảo tính nhất quán
 - D. Kiểm tra tất cả dữ liệu trước khi ghi vào cơ sở dữ liệu

Resilience và triển khai hệ thống

10. Pattern nào không thuộc các chiến lược resilience phổ biến?
 - A. Circuit Breaker
 - B. Bulkhead
 - C. Timeout
 - D. Scale Cube
11. Chiến lược triển khai nào đảm bảo không có thời gian chết khi cập nhật hệ thống?
 - A. Blue-Green Deployment
 - B. Rolling Deployment
 - C. Canary Deployment
 - D. A/B Testing
12. Công cụ nào không được sử dụng để giám sát và theo dõi microservices?

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI ©DNTAI

- A. Prometheus
- B. Elasticsearch
- C. Kubernetes
- D. Grafana

Các bài toán thực tế

13. Khi nào nên sử dụng kiến trúc event-driven?
 - A. Khi cần xử lý dữ liệu thời gian thực và khối lượng lớn
 - B. Khi giao tiếp giữa các dịch vụ đơn giản
 - C. Khi cần đảm bảo tính nhất quán mạnh mẽ
 - D. Khi giao dịch chỉ liên quan đến một dịch vụ
14. Giải pháp nào phù hợp nhất để xử lý vấn đề bottleneck ở cơ sở dữ liệu trong microservices?
 - A. Sử dụng REST API
 - B. Triển khai caching phân tán
 - C. Thêm nhiều dịch vụ vào hệ thống
 - D. Chuyển toàn bộ cơ sở dữ liệu sang NoSQL
15. CQRS giúp ích gì trong kiến trúc microservices?
 - A. Tăng tính đồng bộ của giao tiếp
 - B. Tăng hiệu năng khi xử lý truy vấn và ghi dữ liệu
 - C. Đảm bảo tính nhất quán dữ liệu mạnh mẽ
 - D. Đơn giản hóa thiết kế giao tiếp giữa các dịch vụ

CÂU HỎI LÝ THUYẾT

Kiến thức cơ bản về kiến trúc phần mềm

1. Các kiến trúc phần mềm chính bao gồm những loại nào? So sánh ưu và nhược điểm của chúng.
2. Nguyên tắc KISS, YAGNI, và Separation of Concerns (SoC) có vai trò gì trong thiết kế hệ thống phần mềm?
3. Khi nào nên chọn kiến trúc monolithic và khi nào nên chuyển sang microservices?

Thiết kế và triển khai microservices

4. Mô hình Database-per-Service Pattern là gì? Ưu và nhược điểm của nó trong microservices?
5. Các cách tiếp cận phân mảnh dữ liệu (Data Partitioning) gồm những loại nào?
6. Hãy giải thích các mô hình giao tiếp giữa các microservices như RESTful API, gRPC, và WebSocket. Ưu và nhược điểm của từng mô hình.

Xử lý dữ liệu và giao dịch

7. CQRS là gì? Khi nào nên áp dụng mô hình này?
8. Hãy so sánh SAGA Pattern với các giao dịch truyền thống trong quản lý giao dịch phân tán.
9. Sự khác biệt giữa Event Sourcing Pattern và Transactional Outbox Pattern là gì?

Resilience và triển khai hệ thống

10. Mô hình triển khai nào được sử dụng để giảm thiểu thời gian chết khi triển khai microservices? (VD: blue-green, rolling, canary)
11. Các resilience patterns phổ biến trong microservices bao gồm những gì? Mỗi mô hình giải quyết vấn đề gì?
12. Làm thế nào để giám sát và quan sát microservices sử dụng các công cụ như Prometheus, Grafana, hoặc Elastic Stack?

Các bài toán thực tế

13. Giải pháp nào giúp tăng hiệu năng khi giao tiếp giữa các microservices? (VD: cache phân tán, giảm số lượng API call)

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI ©DNTAI

- 14. Làm thế nào để xử lý bài toán hiệu suất cho ứng dụng cần xử lý hàng triệu sự kiện mỗi giây?
- 15. Khi nào cần sử dụng kiến trúc event-driven thay vì request-response?

Tư duy thiết kế

- 16. Tại sao "mỗi quyết định thiết kế cần được biện minh bởi một yêu cầu kinh doanh" là quan trọng?
- 17. Hãy giải thích khái niệm "Over-engineering" trong thiết kế phần mềm. Làm thế nào để tránh?

--- Hết ---